A single numerical constructor and three inference families for iPRA

Full normalization of counter recursion
with inferred witness opening

Research note prepared for Vladimir Reshetnikov

12 November 2026

Abstract

We give a concrete simplification of the current *ConciseIPRA* presentations. Numerical terms have only variables, zero, and one ternary binding constructor $\mathsf{R}(n, a; i, u.h)$. The same two-variable binding convention serves this constructor and the sole compound formula constructor $\mathsf{D}_{x,y}$. The entire proof calculus has one fixed reflexivity clause and the three inference families Use, Introduce, and Open. Both computation equations are incorporated in canonical syntax.

The construction combines two existing ingredients: the counter recursor of `uniform_binding` and the three-family logic of `inferred_witness`. The new compatibility argument supplies full normalization, an exact translation of the former scalar computation syntax, and a complete checker for their combination. The direct primitive-recursion compiler also shortens the arithmetic bridge: neither pairing nor the scalar predecessor construction is needed. Both interpretations have the whole numerical domain and actual equality; both numerical composites are internally the identity.

The complete matched Python checker decreases from 1,990 to 1,965 normalized bytes and from 771 to 764 AST nodes, with 683 significant tokens in both. This is a measured improvement of a particular complete presentation, not a claim of absolute minimality or a new global byte record. All mathematical claims here have written proofs; no new Lean or Rocq verification is claimed.

Contents

1 Result, comparison, and scope **4**

2 Numerical syntax and full normalization **5**

2.1 One constructor 5

2.2 Canonical forms 5

2.3 A total definition by evaluation 6

3 Primitive recursion is a computational macro **7**

3.1 The decoder 7

3.2 The compiler 7

3.3 Exact compatibility with the scalar syntax 8

4 The entire axiomatization **8**

4.1 Formulas, clauses, and falsity 8

4.2 Use 9

4.3 Introduce 9

4.4 Open with an inferred witness 9

5 Why the three rules suffice **10**

5.1 Soundness of the nonstandard opening 10

5.2 Structural rules and safe openings 10

5.3 Repairing case introduction at witness zero 11

5.4 Substitution and ordinary opening 11

5.5 Equality from a short naming packet 11

5.6 Equation induction 12

5.7 Translation of the inherited normal calculus 12

6 Recovering ordinary intuitionistic logic **13**

7 The ordinary arithmetic realization **14**

7.1 From equation induction to quantifier-free induction 15

8 Both interpretations and both round trips **16**

8.1 Interpreting the new calculus in iPRA 16

8.2 Interpreting iPRA in the new calculus 16

8.3 Numerical composites 16

8.4 Formula composites 17

9 The complete checker and its verification **17**

9.1 Encoding and acceptance contract 17

9.2 The trusted changes 18

9.3 Scope of executable evidence 18

9.4 Measurement 19

10 Further research questions **19**

A Complete acceptance module **20**

B Contents and reproduction **21**

1 Result, comparison, and scope

The comparison is pinned to ProveIt commit

`9f9545f5ffd9e0392d4a36639503aca12383450a`.

The repository contains many presentations, including three-family calculi and several substantially smaller compressed implementations. Consequently, reducing an earlier six- or eight-rule inventory would not by itself improve the current mathematical frontier.

The present construction, denoted $\mathcal{R}_3$, keeps the three-family logical interface and removes the independent successor constructor from its numerical syntax. It also gives every binding constructor exactly two bound positions in its final child. These are changes to the formal language and its computation, not a renaming of Python variables. The actual implementation comparison counts the new normalizer and all validation code.

Theorem 1.1 (Main result)**.** *The calculus $\mathcal{R}_3$ specified in Sections 2 and 4 is equiconsistent and bi-interpretable with full first-order intuitionistic primitive recursive arithmetic. The interpretation domains are the entire numerical domains, interpreted equality is actual equality, and both numerical composite isomorphisms have graph $x = y$.*

There is a direct compiler for every fixed primitive-recursive description into its numerical language. Each compiled recursion has its defining equations by computation. There is also an effective, rule-for-rule translation of the inherited fully normalized scalar inferred-opening calculus into $\mathcal{R}_3$.

For clarity, *full first-order iPRA* here means intuitionistic first-order logic with equality and genuine falsity, the usual successor axioms, the defining equations for every fixed finite primitive-recursive description, and induction for all quantifier-free formulas. Quantified formulas are allowed in contexts and conclusions. Induction for arbitrary quantified formulas is not supplied.

Complete matched checker	Bytes	Tokens	AST nodes
Scalar $0, S, I$ with inferred opening	1,990	683	771
Single-constructor $0, \mathsf{R}$ with inferred opening	1,965	683	764

Both programs include JSON decoding, public syntax and scope checks, normalization, capture-avoiding substitution, every inference instruction, and comparison with the requested conclusion. The differences are 25 bytes (approximately 1.26%) and seven AST nodes, with no token increase. The inherited `uniform_binding` checker uses the same arithmetic idea but five inference families and only zero-body normalization; its published 2,238-byte comparison is a different implementation comparison [2].

What is inherited. The successor-seeded counter recursor, its successor decoder, the direct recursion macro, and the short logical macros below are already in `uniform_binding`. Inferred witness opening and the essential logical completeness argument are already in `inferred_witness` and the associated Lean modules of the repository [3, 4]. This report claims their fully normalized combination, its compatibility proofs, the resulting simpler term grammar, and the matched complete implementation. It does not claim to have invented those ingredients.

What the size result does not say. The repository also supplies a compact 516-byte protocol, a 788-byte checker for a different public grammar, and an 820-byte validator profile for the inherited native calculus [6, 7, 8]. Our 1,965-byte checker is not smaller than those programs. The comparison above isolates the mathematical change while keeping the implementation architecture and complete input contract matched. An observed minimum among programs is an upper bound on the unknown optimum, not a proved lower bound.

2 Numerical syntax and full normalization

2.1 One constructor

Raw terms are finite expressions

$$t ::= x \mid 0 \mid \mathsf{R}(n, a; i, u.h).$$

The two variables i, u are bound in h only. They do not bind in n or a. Passive numerical parameters may occur anywhere they are in scope. Every body is a fixed finite expression; no object-language variable ranges over functions, terms, or programs.

Define successor as an abbreviation:

$$Sa := \mathsf{R}(0, a; i, u.0), \qquad 1 := S0, \qquad 2 := S(S0).$$

The intended computations are

$$\mathsf{R}(0, a; i, u.h) = Sa, \tag{1}$$

$$\mathsf{R}(Sn, a; i, u.h) = h[n/i, \mathsf{R}(n, a; i, u.h)/u]. \tag{2}$$

Thus a zero iteration returns the successor of the initializer. The update sees both the old counter and the old state. The use of a successor initializer is precisely what allows S to be expressed with the same constructor.

These two equations specify computation, rather than additional proof rules. Every term appearing in a sequent is in the canonical form described next, and every substitution is followed by normalization. This convention is part of the trusted specification.

2.2 Canonical forms

A canonical successor is exactly $\mathsf{R}(0, a; i, u.0)$, with a canonical. A canonical term other than zero or a canonical successor is called *neutral*. Neutrals are variables and recursor nodes with a neutral counter. In a retained recursor node its initializer and body are also canonical.

Operationally, canonicalization erases the body of every zero-counter node, and unfolds every successor-counter node using (2). A node already of the form $\mathsf{R}(0, a; i, u.0)$ is retained; the first computation is not applied as a self-loop. The normalizer acts under binders as well as at the root.

The zero-body erasure does not license malformed input. The supplied checker validates the raw body and its scope before erasing it. For example, a positive index outside the two new bound positions cannot be hidden in a discarded body.

2.3 A total definition by evaluation

Here is a definition of the normalizer independent of any proposed termination measure on rewrite sequences. An environment ρ maps free variables to normal terms. Evaluate zero and variables directly. To evaluate $\mathsf{R}(n, a; i, u.h)$, evaluate n, a to c, v and use the already defined evaluator of its proper subterm h:

$$H(d, w) = \mathrm{ev}(h, \rho[i \mapsto d, u \mapsto w]).$$

For that fixed evaluator define

$$\mathrm{run}(c, v, H) = \begin{cases} Sv, & c = 0, \\ H(d, \mathrm{run}(d, v, H)), & c = Sd, \\ \mathsf{R}(c, v; i, u.H(i, u)), & c \text{ neutral}. \end{cases} \tag{N}$$

In the neutral case, i, u are fresh and the environment is lifted past them. The notation $H(i, u)$ means evaluation in this extended environment; it introduces no function object into the numerical language. Put $\mathrm{N}(t) = \mathrm{ev}(t, \mathrm{id})$.

Lemma 2.1 (Normalization and substitution)**.** *Evaluation is total on finite terms and finite normal environments. Its result is canonical. It fixes every canonical term, respects* (1)–(2)*, and satisfies*

$$\mathrm{N}(t\sigma) = \mathrm{N}\big(\mathrm{N}(t)\ \mathrm{N}(\sigma)\big). \tag{SC}$$

The same identity holds for formulas when their terms are normalized. In particular, normalization is idempotent and commutes with fresh renaming.

Proof. Define evaluation by structural recursion on the source expression. At a recursor node, the evaluators of its counter, initializer, and body have already been defined for all normal environments. For a fixed total body evaluator, (N) strips the finite leading successor prefix of the normal counter. Its only recursive call to the same run has the strictly shorter prefix. Applying H after that call is total by the structural induction hypothesis. The neutral case evaluates the proper subterm h under two fresh variables. This establishes totality and normality.

The algebraic verification is made with arbitrary environments. Renaming the target commutes with evaluation; renaming the source is precomposition of the environment. Evaluation after substitution equals evaluation in the environment obtained by evaluating the substitution's values. These statements are proved by structural induction, with the three clauses of (N) as the additional cases. At zero both sides are the successor of the transformed initializer. At a successor, use the statement for the smaller counter and for the body with the two transformed arguments. At a neutral counter, the residual body is the same body evaluator under two fresh variables. If substitution turns that counter into zero or a successor, evaluating the residual node uses exactly the corresponding clause of (N). Thus the statement also covers newly exposed redexes.

In the identity environment a normal term is fixed by the cases of its grammar. Environment composition gives (SC) and idempotence. The first two clauses of (N) establish the computation identities in every environment. Each recursive evaluation also gives a finite conversion using those identities, including erasure of an unused zero body. Thus equal terms in the computational congruence have the same canonical representative. When this congruence is read in ordinary first-order arithmetic, its use under a recursor body is justified by Lemma 7.1 below. □

No uniform primitive-recursive bound on normalization time is claimed. The construction is a metatheoretic operation on fixed finite syntax. Ordinary implementation resource limits still apply to the executable checker.

3 Primitive recursion is a computational macro

3.1 The decoder

Define

$$P(t) := \mathsf{R}(t, 0; i, u.i).$$

Its two relevant computations are

$$P(0) \longrightarrow 1, \qquad P(Sa) \longrightarrow a. \tag{P}$$

The value at zero is intentionally 1, not the usual predecessor value. The second identity holds for every open term a by computation: the body returns the old counter and ignores the old state. No injectivity or induction argument is involved.

3.2 The compiler

For an arbitrary fixed update body $h(i, v)$, define

$$\mathrm{Rec}(n, a; i, v.h) := P\big(\mathsf{R}(n, a; i, u.S(h(i, P(u))))\big). \tag{3}$$

All bound names are chosen fresh and the indicated substitution is capture avoiding. The initializer a may contain passive parameters, as may h.

Lemma 3.1 (Ordinary recursion equations)**.** *The following are computational identities:*

$$\mathrm{Rec}(0, a; i, v.h) \longrightarrow^* a, \tag{4}$$

$$\mathrm{Rec}(Sn, a; i, v.h) \longrightarrow^* h(n, \mathrm{Rec}(n, a; i, v.h)). \tag{5}$$

Proof. At zero the inner recursor becomes Sa, and $P(Sa)$ reduces to a. At a successor the inner recursor becomes

$$S\big(h(n, P(\mathsf{R}(n, a; i, u.S(h(i, P(u)))))\big)\big).$$

The outer P removes this displayed successor. What remains is the right side of (5). □

Zero, successor and projections are now terms; composition is substitution; and an ordinary primitive-recursion clause is translated by (3). External induction on a finite primitive-recursive description therefore compiles every primitive-recursive function. In particular, no pairing function is needed to store the old counter beside the old state.

The ordinary predecessor and the selector can be chosen as

$$p(t) := \mathrm{Rec}(t, 0; i, v.i), \qquad C(t, a, b) := \mathrm{Rec}(t, a; i, v.b).$$

They have the computational equations

$$p(0) = 0, \quad p(Sa) = a, \qquad C(0, a, b) = a, \quad C(Sn, a, b) = b. \tag{C}$$

The symbol p is distinct from the decoder P. For example, addition is $\mathrm{Rec}(n, a; i, v.Sv)$ and multiplication is $\mathrm{Rec}(n, 0; i, v.(v + a))$, after expanding the already compiled addition. These are term abbreviations, not new primitives.

3.3 Exact compatibility with the scalar syntax

The inherited scalar term language has $0, S$ and $I(n, a; v.h)$, where the update does not see the counter. Translate its variables and zero identically, its successor by the abbreviation S, and its iterator by the counter-independent instance of (3). Write T_{raw} for this hygienic translation before normalization, and T for the translation on normal source terms.

Lemma 3.2 (Exact normal-form embedding)**.** *For every scalar term t,*

$$\mathrm{N}_R(T_{\mathrm{raw}}t) = T(\mathrm{N}_I\, t).$$

Raw translation commutes with capture-avoiding substitution; normalized translation commutes with normalized substitution. On normal source terms it preserves and reflects literal zero, each specified free variable, and the successor category. It also preserves their free-variable sets.

Proof. The two scalar computation equations translate to the finite computations in Lemma 3.1. A normal scalar iterator has a neutral counter. Its image is

$$P\big(\mathsf{R}(Tn, Ta; i, u.S((Th)[P(u)/v]))\big).$$

Here Tn is neutral. Thus the inner recursor is retained and neutral, and the outer P is retained as well. Substitution of the neutral term $P(u)$ for the old body variable cannot turn a neutral counter into zero or a successor. Structural induction therefore proves normality of the whole image. Zero, variables and successors give the remaining cases.

The raw substitution lemma follows by induction on terms, freshening the two new variables at every translated iterator. Lemma 2.1 then gives the normalized version and the displayed equality. The four source categories have distinct target root descriptions: zero, the same variable, a canonical successor, or a neutral recursor. No free variable in a normal source term is erased by this construction. □

This is more than a comparison of standard numerical values. It preserves the literal row tests used by the proof rules below. In particular, a row becomes $0 = 0$, or has a specified fresh variable as its entire left side, exactly when its normal source row did.

4 The entire axiomatization

4.1 Formulas, clauses, and falsity

The formula grammar is

$$A ::= (s = t) \mid \mathsf{D}_{x,y}B.$$

Both x, y are bound in B. There are no primitive propositional connectives or ordinary quantifiers. Terms and formulas are canonical, and displayed substitutions include normalization. The designated contradiction is

$$\bot_* := (0 = S0).$$

A context is a finite list of reusable clauses

$$[\,\Delta\,;\, y.B(y)\,], \qquad |\Delta| \in \{0, 1\}, \qquad y \notin \mathrm{FV}(\Delta).$$

An empty antecedent has no proof obligation. An ordinary assumption A is the constant unconditional clause $[\,;\, v.A\,]$ with v unused. Every proof starts with the fixed clause

$$\mathcal{A} := [\,;\, y.y = y\,]. \tag{Ax}$$

It is part of the axiomatization. Reflexivity has not been deleted without replacement.

The ordinary readings, used to state soundness and bi-interpretability, are

$$(s = t)^\circ = (s = t), \tag{6}$$
$$(\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big), \tag{7}$$
$$[\,;\, y.B\,]^\circ = \forall y\, B^\circ, \tag{8}$$
$$[\,A\,;\, y.B\,]^\circ = A^\circ \to \forall y\, B^\circ. \tag{9}$$

4.2 Use

For a retained clause $[\,\Delta\,;\, y.B(y)\,] \in \Gamma$ and any well-scoped canonical term t,

$$\frac{\{\Gamma \vdash A :\ A \text{ is the antecedent in } \Delta\}}{\Gamma \vdash \mathrm{N}(B(t))}. \tag{Use}$$

Thus an unconditional clause has zero premises, and a conditional clause has one. Applying the fixed clause $\mathcal{A}$ gives $t = t$.

4.3 Introduce

For a well-scoped canonical witness t and a fresh numerical parameter z,

$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{Intro}$$

The parameter z is absent from the retained context, the witness, and the passive parameters of B. The assumption displayed above the line is discharged. The test concerns literal canonical syntax, not provable equality. The first branch is progress, and the second is case introduction.

4.4 Open with an inferred witness

Suppose the major premise is $\mathsf{D}_{x,y}B(x,y)$. Choose b fresh for its passive parameters and the retained context. Put $B_b(y) = \mathrm{N}(B(b,y))$ and

$$\tau_B(b) = \begin{cases} r, & B_b(0) \equiv (0 = 0) \text{ and } B_b(1) \equiv (b = r), \\ b, & \text{otherwise.} \end{cases} \tag{Probe}$$

The fresh variable must be the *entire* left side of the first row. No search for a provable equality takes place. The rule is

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B \qquad \Gamma, [\,B(\tau_B(b),0)\,;\, y.B(\tau_B(b),y)\,] \vdash A}{\Gamma \vdash A}, \qquad b \notin \mathrm{FV}(A). \tag{Open}$$

All substituted instances are normalized. The branch is checked in the scope that contains b, even if the probe has chosen a term containing b. Replacement is simultaneous and occurs once; inserted copies are not revisited.

These three families, Ax, and the specified canonical syntax are the complete presentation. There are no additional primitive equality, computation, induction, hypothesis, cut, or ordinary logical rules. Structural operations used in the mathematical proofs below are admissibility theorems.

5 Why the three rules suffice

This section makes the logical dependency order explicit. The argument is inherited in structure from `inferred_witness` [3]; it applies to arbitrary R terms because the selector equations (C) are computational identities in the new language.

5.1 Soundness of the nonstandard opening

In ordinary intuitionistic equality logic, open the existential major at b and obtain $B(b,0) \to \forall y B(b,y)$. If the probe succeeds, reflexivity supplies $B(b,0)$ and specialization at 1 gives $b = \tau_B(b)$. If it fails, this equality is $b = b$. Equality substitution converts the opened clause to the clause displayed in Open. The branch then supplies A, which does not contain b, so existential elimination closes. This also proves soundness when the inferred term contains b.

Case Intro is sound using numerical coverage. Progress Intro is sound using equation induction under its conditional zero base. Use and Ax have their ordinary meanings. Thus every rule is sound in the ordinary realization introduced in Section 7.

5.2 Structural rules and safe openings

Weakening by clause occurrences, injective fresh renaming, and cut of an ordinary assumption are admissible by induction on derivations. At an assumption cut, replace uses of its unconditional constant clause by the given proof. Freshen local parameters before insertion. Injective renaming preserves the literal tests. Arbitrary term substitution is not yet being asserted.

Open is already ordinary existential-style elimination in two cases: when the witness coordinate is unused in B, and when B has compound root. In the first case the first row contains no fresh b, so its left side cannot be literally b. In the second case the zero row is still compound and cannot be $0 = 0$. Hence the probe returns b.

For an unused witness coordinate write $q_y A(y) := \mathsf{D}_{w,y} A(y)$. Introducing its witness at 1 always selects case mode. The two safe openings therefore give

$$\frac{\Gamma, A(0) \vdash A(Sz)}{\Gamma \vdash q_y A(y)}, \qquad \frac{\Gamma \vdash q_y A(y) \quad \Gamma \vdash A(0)}{\Gamma \vdash A(t)}. \tag{Closure}$$

In particular, from proofs of $A(0)$ and $A(Sz)$ with fresh z, numerical cases yield $A(t)$. This fact is available *before* equality transport.

5.3 Repairing case introduction at witness zero

An arbitrary substitution can send a nonzero witness to literal zero and switch Intro's mode. The following repair is required before any substitution theorem.

For an equation $E = (s = t)$ put

$$G_v E := (C(v, 0, s) = C(v, 0, t)), \qquad L(E) := q_v G_v E.$$

The endpoints are literally $G_0 E = (0 = 0)$ and $G_{Sw} E = E$. Closure introduction and specialization give $E \dashv\vdash L(E)$, using only safe openings.

Suppose $\Gamma, E(0,0) \vdash E(0, Sz)$, the missing case premise for an equation at witness zero. Replace its matrix by the compound formula $L(E(x, y))$. Unwrap the base, apply the supplied proof, and wrap the successor result. Compound Intro at zero is in case mode, so it gives $\mathsf{D}_{x,y} L(E(x, y))$. Open that compound major safely at a fresh c. Introduce $\mathsf{D}_{x,y} E(x, y)$ at witness c: wrap its zero assumption, use the opened clause at Sz, and unwrap. The fresh variable c is not literal zero, so this last introduction is also in case mode. Its conclusion is independent of c and is exported.

Thus unrestricted case introduction is admissible for every matrix and every witness. No general substitution or equality theorem has been used in its proof.

5.4 Substitution and ordinary opening

Lemma 5.1 (Admissible simultaneous substitution)**.** *If* $\Gamma \vdash A$*, then* $\mathrm{N}(\Gamma\sigma) \vdash \mathrm{N}(A\sigma)$ *for every well-scoped simultaneous numerical substitution* σ*.*

Proof. Use strong induction on the original derivation, with the induction assertion quantified over all substitutions. Freshen its eigenparameters away from every substitution image. Use follows from Lemma 2.1. A progress Intro remains progress. A case Intro remains a case unless its equation witness becomes zero; use the preceding case repair then.

At an Open node, let $B' = \mathrm{N}(B\sigma)$ and $\tau' = \tau_{B'}(b)$. If the old probe succeeded at r, its literal zero row and its row $b = r$ remain such rows after substitution with b fixed. Hence the new probe succeeds at $\mathrm{N}(r\sigma)$; translate the old branch with b fixed. If the old probe failed, its branch had the ordinary clause at b. The new probe might now succeed. Apply the induction hypothesis to the original branch with the combined substitution extending σ by $b \mapsto \tau'$. It produces exactly the new opened clause. The retained context and conclusion originally omitted b, so they are unchanged by this extra component. Inserted images are not recursively substituted into themselves. The induction is always on an original proper subderivation, which avoids circular use of a transformed, larger proof. □

Ordinary paired elimination is now admissible: given a branch under the ordinary clause at b, apply Lemma 5.1 to that branch with $b \mapsto \tau_B(b)$, then use Open. Its outer conclusion and context omit b. This derivation is why the substitution theorem must precede unrestricted opening.

5.5 Equality from a short naming packet

Lemma 5.2 (Transport in a term)**.** *For every fixed numerical template* $h(w)$*,*

$$\Gamma \vdash s = t, \quad \Gamma \vdash h(s) = r \quad \Longrightarrow \quad \Gamma \vdash h(t) = r.$$

Proof. Use the canonical matrix

$$B(x,y) := C(y,0,C(P(y),x,h(x))) = C(y,0,C(P(y),t,r)). \tag{10}$$

Its normal rows are

$$B(x,0) \equiv (0=0), \quad B(x,1) \equiv (x=t), \quad B(x,2) \equiv (h(x)=r).$$

Moreover its arbitrary successor row is literally

$$B(x,Sz) \equiv C(z,x,h(x)) = C(z,t,r),$$

because $P(Sz)$ computes to z. At witness s, numerical cases on z prove this equation: the zero case is the supplied $s=t$, and the successor case is the supplied $h(s)=r$. The proof ignores Intro's local assumption, so it works in either mode and introduces $\mathsf{D}_{x,y}B$.

Open infers the witness t. Its clause has base $0=0$. Reflexivity supplies that base, and Use at 2 gives $h(t)=r$, which is in the outer scope. □

The decoder makes this packet simpler than the parity packet in the scalar inferred-opening presentation: one successor computation exposes the very variable on which numerical cases are taken.

Symmetry follows by taking $h(w)=w$ and the reflexive minor $s=s$. Transitivity and congruence follow by the usual finite applications. Equality replacement in both sides of an equation takes two one-term replacements. For a compound formula template, use induction on its formula depth: open the source at a fresh witness, introduce the target in case mode at that witness, transport its base backward in the smaller matrix, use the old clause at a successor, and transport forward. Thus full formula transport is derived, not primitive.

5.6 Equation induction

Given $\Gamma \vdash E(0)$ and $\Gamma, E(z) \vdash E(Sz)$ for a literal equation E, Intro at witness zero with unused witness coordinate gives $q_y E(y)$. Its opening is safe. Use with the given base yields $E(t)$ for every term t. This derives contextual equation induction. No corresponding progress rule is allowed at compound matrices.

5.7 Translation of the inherited normal calculus

The term translation of Lemma 3.2, extended without changing equation or D roots, translates each primitive instruction of the inherited fully normalized inferred-opening calculus. At Use, this follows from commutation with normalized substitution. At Intro, literal-zero reflection preserves the mode, and the translated successor row is the required successor row. At Open the two row tests are preserved and reflected, so

$$\tau_{T(B)}(b) = T(\tau_B(b)).$$

Normalized substitution then gives exactly the translated branch clause. Free-variable preservation supplies the scope conditions in all three cases.

This proves the rule-for-rule assertion of Theorem 1.1 for the *already normalized* source calculus. If one starts with raw scalar proofs, their prior normalization can turn a witness into zero; the admissible case repair above handles that separate normalization step.

6 Recovering ordinary intuitionistic logic

The following definitions are external operations on finite formula syntax. They introduce no new primitive connectives or inference rules.

Extend the gate from equations to all formulas by

$$G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

freshening bound variables away from the control v. Reflexivity, compound introduction, and induction on formula depth give

$$\vdash G_0A, \qquad G_{Sz}A \dashv\vdash A, \qquad A \dashv\vdash q_yG_yA \quad (y \notin \mathrm{FV}(A)). \tag{Gate}$$

Define a selector between arbitrary formulas, first at equations:

$$M_v(s = t, r = w) := (C(v, s, r) = C(v, t, w)).$$

Align fresh binders at compound roots and use

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &= \mathsf{D}_{x,y}M_v(A, G_yE). \end{aligned}$$

Here E is an equation. Each recursion reduces the maximum binder depth, so the definition terminates. Gate padding and binder congruence establish

$$M_0(A, B) \dashv\vdash A, \qquad M_{Sz}(A, B) \dashv\vdash B.$$

The complete ordinary logical interface is

$$\begin{aligned} \bot_* &:= (0 = S0),\\ A \Rightarrow_* B &:= q_yM_y(A, B),\\ A \wedge_* B &:= q_yG_yM_{P(y)}(A, B),\\ \exists_*x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_*x\, M_x(A, B),\\ \forall_*x\, A(x) &:= q_yG_yA(P(y)). \end{aligned} \tag{11}$$

All new bound names are fresh. The witness coordinate of q is unused.

Implication. From a derivation of B under A, case introduction uses the zero endpoint A and the successor endpoint B. Elimination opens the clause and uses it at 1.

Conjunction. From A, B, numerical cases give $M_z(A, B)$. Since $P(Sz)$ computes to z, this is exactly the successor gate required for introduction. For projections, open its zero-gated clause and use it at 1 and 2; the results are respectively A and B.

Existential quantification. Introduce with the given numerical witness and the successor gate. Eliminate at a fresh witness, supply the zero-gate theorem, and use the clause at 1. These are precisely the ordinary eigenvariable conditions.

Disjunction. The two injections use witnesses 0 and 1 in the existential macro. For elimination, discharge the two branches as implications into the desired conclusion. Numerical cases show that either selector endpoint implies that conclusion. Existential elimination then completes the proof.

Universal quantification. From a proof of $A(z)$ with fresh z, case introduction of the unused witness at 1 supplies $q_y G_y A(P(y))$: its successor row is $A(z)$ by computation. For elimination at an arbitrary term t, open the unused witness, supply the zero gate, and use the clause at St. The result is $A(P(St)) = A(t)$ by computation. There is no extra zero-instance conjunct in this macro. The decoder makes the elimination work uniformly also at $t = 0$.

Falsity. Under $0 = 1$, equality transport gives $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary terms. Every equation follows. External induction on formula depth, using case introduction at compound roots, gives every formula. This is falsity elimination. No excluded middle for quantified formulas has been introduced.

Coverage. Numerical cases prove

$$(x = 0) \vee_* \exists_* v\, (x = Sv).$$

At zero use the left injection of reflexivity; at a successor use the right injection with its predecessor as witness. Applying the ordinary predecessor equation to a successor witness strengthens this to $x = 0 \vee_* x = S(p(x))$.

7 The ordinary arithmetic realization

Let T_{R} be intuitionistic first-order equality logic in the staged R language, with genuine falsity, computations (1)–(2), coverage, the separation axiom $(0 = S0) \to \bot$, and contextual induction for literal equations. A staged symbol is used for each fixed finite body; it never quantifies over descriptions.

Sections 5–6 prove that $\mathcal{R}_3$ and T_{R} have the full logical interface in both directions. Soundness was checked rule by rule. Conversely, the macros derive each ordinary logical rule, coverage, equation induction, and the translation of separation. Computational equality is already canonical equality. The separation axiom is explicit in the ordinary realization; equations alone would admit a one-element structure.

Lemma 7.1 (Body extensionality)**.** *In T_{R}, if $a = b$ and the two fixed bodies satisfy $h(i, u) = k(i, u)$ for arbitrary i, u, then*

$$\mathsf{R}(n, a; i, u.h) = \mathsf{R}(n, b; i, u.k)$$

for every n. The premise may be a retained universally quantified assumption; no function quantification is used.

Proof. Induct on the displayed literal equation with a fresh counter variable, keeping all passive parameters fixed, then instantiate that variable at n. At zero its two sides are Sa, Sb, equal by congruence. At a successor, the induction hypothesis identifies the two previous states. Instantiate the pointwise body equality at the old counter and one state, and use ordinary equality congruence for the other state. This is the required step. □

Consequently, replacing a fixed body by its normalized form is sound in the ordinary first-order realization. The same lemma justifies using a provably equal returned body, rather than a literally identical one, in the interpretation round trips.

7.1 From equation induction to quantifier-free induction

For completeness we give the arithmetic argument, adapted from the inherited equality-reflection proof [5]. All functions in it are direct Rec macros by Section 3. Thus their availability is not contingent on quantifier-free induction.

From $Sa = Sb$, applying P gives $a = b$. Put $\mathrm{sg}(v) = C(v, 0, 1)$. Applying this term to $0 = Sa$ yields $0 = 1$, so successor is separated from zero. Coverage gives

$$\mathrm{sg}(v) \in \{0, 1\}, \quad \mathrm{sg}(v) = 0 \;\leftrightarrow\; v = 0.$$

Here $t \in \{0, 1\}$ abbreviates $t = 0 \vee t = 1$. Let $K(v) = C(v, 1, p(v))$. Then $K(0) = 1$, $K(Sv) = v$, and coverage yields

$$K(v) = 0 \;\leftrightarrow\; v = 1.$$

These are constructive case arguments.

Define addition and truncated subtraction by

$$\begin{aligned} x + 0 &= x, & x + Sy &= S(x + y), \\ d(x, 0) &= x, & d(x, Sy) &= p(d(x, y)). \end{aligned}$$

Four inductions on literal equations give

$$0 + y = y, \quad (Sx) + y = S(x + y), \quad d(Sx, Sy) = d(x, y), \quad d(Sa, a) = 1. \tag{A}$$

For the first two, induct on y and compute. For the third the base is $p(Sx) = x$ and the step applies p to the induction equality. For the fourth the base is $d(1, 0) = 1$ and its step is the third identity followed by the induction hypothesis.

The crucial further induction is on the *single equation*

$$C(\mathrm{sg}(d(x, y)), x, y + d(x, y)) = x. \tag{12}$$

At $y = 0$, its two selector alternatives are both x, using $0 + x = x$. For the step set $r = d(x, y)$ and $r' = p(r) = d(x, Sy)$. If $r' = 0$, the new selector returns x. Otherwise coverage gives $r' = S(p(r'))$. The zero case for r is contradictory, and its successor case gives $r = Sr'$. The induction equation therefore says $y + r = x$. The identity $(Sy) + r' = y + Sr' = y + r$ from (A) proves the new selected equality. All case splits use coverage, not induction on an implication.

Substitute $x = Sa, y = b$ in (12). If $d(Sa, b) = 1$, then $b + 1 = Sa$, so $Sb = Sa$ and $b = a$. Conversely $a = b$ and (A) give that residual value. Hence

$$d(Sa, b) = 1 \quad \leftrightarrow \quad a = b.$$

The fixed term

$$e(a, b) = \mathrm{sg}(K(d(Sa, b)))$$

therefore satisfies, throughout the numerical domain,

$$e(a, b) \in \{0, 1\}, \qquad e(a, b) = 0 \leftrightarrow a = b. \tag{Equality reflection}$$

In particular, equality is decidable; this was proved rather than assumed.

For a fixed quantifier-free formula A, define its characteristic term by finite external recursion:

$$\begin{aligned} b_{s=t} &= e(s,t), & b_{\bot} &= 1, \\ b_{A\wedge B} &= C(b_A, b_B, 1), & b_{A\vee B} &= C(b_A, 0, b_B), \\ b_{A\to B} &= C(b_A, b_B, 0). \end{aligned}$$

Constructive cases on the two-value characteristics prove $b_A \in \{0,1\}$ and $b_A = 0 \leftrightarrow A$. For implication, the $b_A = 0$ branch has A and reduces to b_B; the $b_A = 1$ branch has $\neg A$ and reduces to zero. This uses decidability of a fixed quantifier-free formula, never excluded middle for arbitrary quantified formulas.

Translate a quantifier-free induction premise through this equivalence, apply equation induction to $b_A(n) = 0$, and translate back. The retained context can contain arbitrary formulas as long as the usual induction-variable freshness holds. Thus T_{R} recovers the full quantifier-free induction schema of iPRA.

8 Both interpretations and both round trips

8.1 Interpreting the new calculus in iPRA

In iPRA, interpret every fixed recursor body by ordinary primitive recursion with initializer Sa:

$$r_h(0, a, \vec{p}) = Sa, \qquad r_h(Sn, a, \vec{p}) = h(n, r_h(n, a, \vec{p}), \vec{p}).$$

Interpret D and clauses by (7). Every new rule is sound by the argument of Section 5. Canonicalization preserves these interpretations because its computations are the displayed defining equations.

The interpretation domain is $x = x$. No standardness predicate, coding range, retract or quotient is introduced. Equality is unchanged.

8.2 Interpreting iPRA in the new calculus

Translate each fixed primitive-recursive description by (3), and each ordinary logical operation by (11). The primitive-recursive defining equations compute. The successor axioms, ordinary logical rules, and quantifier-free induction were derived above. This therefore interprets the entire first-order theory, not merely its equational fragment.

8.3 Numerical composites

For a fixed recurrence, two functions with the same initial value and update are provably equal by contextual equation induction: the base is their common initializer and the step uses congruence in the common update.

External induction on a finite primitive-recursive description therefore shows that its translation into R and back equals the original operation at every internal argument.

The other direction must check the primitive R itself, not only translated scalar iterators. Interpreting R in iPRA and compiling back, then identifying the returned initializer and body by induction on

their finite syntax, gives a term provably equal to

$$\mathrm{Rec}(n, Sa; i, v.h(i, v)).$$

It has the same initial value Sa and the same successor update as $\mathsf{R}(n, a; i, u.h(i, u))$. Equation induction proves

$$\mathrm{Rec}(n, Sa; i, v.h(i, v)) = \mathsf{R}(n, a; i, u.h(i, u)). \tag{13}$$

An induction on term construction now gives the other numerical round trip. Consequently $x = y$ is a definable total bijective identity graph preserving every operation in both composites.

8.4 Formula composites

The essential native identity is

$$\mathsf{D}_{x,y}B(x, y) \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{14}$$

Forward, ordinarily open the left side at b. Under $B(b, 0)$ its clause supplies $B(b, z)$ for a fresh z. Introduce universal, implication and existential macros, and close the opening. Backward, eliminate the existential macro at a fresh witness and introduce $\mathsf{D}B$ in case mode at that witness. Its zero assumption, the implication, and universal elimination supply the successor instance. Close the outer existential elimination.

The interpretations of all ordinary macros have their ordinary round trips: selector endpoints give implication and disjunction; rows $1, 2$ give conjunction; the zero gate and row 1 give existential quantification; and row St with $P(St) = t$ gives all universal instances. Falsity uses the explicit separation bridge. External induction on formula syntax, using (13) and (14), proves both formula composites.

In particular,

$$\mathcal{R}_3 \vdash 0 = S0 \quad \Longleftrightarrow \quad \mathsf{iPRA} \vdash \bot.$$

This is equiconsistency and completes the proof of Theorem 1.1. It is a metatheorem, not an assertion that either arithmetic proves its own consistency.

9 The complete checker and its verification

9.1 Encoding and acceptance contract

The source is `code/checker.py`. It needs only Python's standard library. Its raw integer/list encoding is:

Object	JSON representation
Zero	`0`
Free numerical parameter	negative integer level
Bound variable	positive de Bruijn index
$\mathsf{R}(n, a; i, u.h)$	`[n,a,h]`
$s = t$	`[s,t]`
$\mathsf{D}_{x,y}B$	`[B]`

Inside a recursor body, index 1 is u and index 2 is i. Inside a D matrix, index 1 is y and index 2 is x. Both constructors add two bound positions in their last child. A successor is `[0,a,0]`.

An input is a JSON object with exactly the keys `goal` and `proof`. Its goal is closed and canonical. The proof instructions are

```
['u', clause_index, term, premise],
['i', matrix, witness, branch],
['e', major, branch].
```

For an unconditional clause, the premise payload is ignored; use JSON `null`. There are no separate proof instructions for equality or computation. Acceptance means normal process exit with `ACCEPT`; malformed syntax, an incorrect proof, or an exception does not constitute acceptance.

9.2 The trusted changes

Relative to the pinned inherited checker, the changes are confined to the arithmetic syntax, the shared binder increment, the recursor computation, and the successor abbreviations in the rules. The three proof instructions and their logical tests are unchanged.

Because R is the only compound numerical constructor, a compound canonical counter is a successor exactly when its first child is zero. The old test distinguishing a two-child S node from a three-child I node is unnecessary. This is the reason the new syntax reduces validator and normalizer code despite the update's extra counter argument.

When unfolding $\mathsf{R}(Sn, a; i, u.h)$, the implementation first substitutes the old counter at depth 1, preserving the state slot, and then substitutes the recursive state at depth 0. This order preserves outer indices under nested binders. Both substitutions use the same capture-avoiding operation.

9.3 Scope of executable evidence

Proposition 9.1 (Checker correspondence)**.** *Ignoring finite physical resource limits, the acceptance module accepts exactly the closed canonical conclusions having a finite $\mathcal{R}_3$ derivation, through the stated certificate encoding.*

Proof. On well-scoped finite syntax, induction on the tree shows that the shared traversal implements the constructor and binder conventions. Its recursor case implements (N); the additional induction is on the finite successor prefix of its counter. The variable callback in `syntax` checks precisely the free-level range and local bound depth, and comparison with the result enforces canonicality. Every child is checked before a zero-body erasure.

The callback in `sub` replaces the specified bound slot, shifts the inserted expression past inner binders, and lowers the outer indices after removing the slot. Tree induction, together with (SC) when a substitution exposes a recursor redex, proves capture-avoiding opening followed by normalization.

Induct now on an accepting proof-call tree. Use checks the selected index and its antecedent when present. Introduce checks exactly its successor row under the chosen assumption and validates the original matrix in the returned formula. Open checks its major before extending the context, computes the specified probe, and validates the branch conclusion again in the outer scope. If its probe succeeds, the unmodified stored base is $0 = 0$, so it also equals the substituted base required by the mathematical rule. These are exactly the three inferences.

Conversely, decorate a finite derivation with the retained clause indices, its matrices and witnesses, and fresh negative levels in order of scope creation. Use `null` for an unused premise. Induction on the derivation shows that the module validates this certificate. The global comparison checks its closed goal. No test-generator correctness assumption enters either direction. □

The accompanying reproduction program checks the following concrete risks:

- 13 positive proof certificates, covering all three rules, both introduction modes, inferred and fallback openings, nested binders, compound formulas, and an equation induction that actually uses its induction hypothesis;
- 21 rejection controls, including escaping eigenparameters, wrong clause antecedents, malformed matrices, noncanonical terms, and invalid syntax hidden in an erased body;
- normal and optimized Python command-line replay of the positive and negative cases;
- 600 independently evaluated normalization/idempotence cases, 600 partial-substitution checks retaining a bound state coordinate, and 600 scalar-to-R compiler checks;
- fifteen symbolic reduction checks, including open-term decoder and initialized-recursion identities.

The direct arithmetic builders also normalize $3 + 4$, $3 \cdot 4$, the fifth triangular number, and $4!$ to $7, 12, 15, 24$, respectively.

The randomized checks have a fixed seed and finite evaluation cutoffs recorded in the reproduction program. They are tests of implementations, not a proof of universal soundness, normalization, or bi-interpretability. Those claims are supplied by the written arguments. The archive does not represent this new calculus as Lean-verified.

9.4 Measurement

The reference runtime is CPython 3.12.14. For each entire acceptance module, the reproduction program parses the source, applies `ast.unparse`, and appends one LF. It counts UTF-8 bytes, significant Python tokens (excluding comments, whitespace and layout tokens), and all AST nodes. Neither checker contains a docstring, so docstring removal cannot affect the matched comparison.

The inherited source is included unchanged, with its pinned path and SHA-256 in the provenance file. The proof compiler and test generators are untrusted support tools: their output is always checked by the standalone checker. Their size is not included in the proof-checker metric, just as proof-search and certificate generation are excluded from the matched baseline.

10 Further research questions

1. **Formal verification of the new normalization layer.** Extend the repository's normalization by evaluation development from scalar I to counter-aware R, prove the environment laws, and lift the existing inferred-opening comparison through the exact embedding in Lemma 3.2. This is the most direct next verification step.
2. **A smaller shared binder-opening operation.** Both object-language constructors now bind two variables, but retained clauses have one universal variable. Can a uniform representation

remove partial binder opening while reducing the *whole* checker, including validation and scope checks?

3. **Remove the witness test without adding a larger grammar test.** Guarded atomic matrices can make introduction depend only on the formula root and can guarantee a reflexive zero row for opening. The open issue is whether the resulting complete grammar and checker are simpler than the unrestricted $\mathcal{R}_3$ presented here.
4. **A comparison below the compressed frontier.** Port this arithmetic/logic combination into the repository's compact certificate architecture and count every validator and normalizer. The present 1,965-byte result does not settle whether that can beat the substantially smaller existing profiles.
5. **Resource-controlled checking.** A fuel-bounded implementation with explicit rejection can make runtime limits part of the certificate contract. Compare such implementations with equally bounded controls rather than charging this overhead to only one side.
6. **Meaningful minimality statements.** A global claim needs a fixed description language, permitted runtime primitives, and input contract. Can a lower bound be proved in a useful restricted class of proof checkers, rather than inferred from the smallest program currently found?

A Complete acceptance module

The following is the complete `code/checker.py`. No external theorem library, normalizer, or validator is called.

```
import json

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or (size in (1, 2) if form else size == 3))
    a = [walk(b, f, d + 2 * (size == 1 or i == 2), size == 1) for i, b in enumerate(a)]
    if size == 3:
        c, b, h = a
        if c == 0:
            return [0, b, 0]
        if type(c) is list and c[0] == 0:
            return sub(sub(h, c[1], 1), walk([c[1], b, h], lambda i, d: i))
    return a

def syntax(a, n, form=False):
    need(walk(a, lambda i, d: need(1 - n <= i <= d) or i, form=form) == a)

def sub(a, t, depth=0):
    return walk(a, lambda i, d: walk(t, lambda j, e: j + d if j > e else j) if i == d + 1 else i - (i > d +
     1), depth)

def prove(p, g):
    n = len(g)
    b = -n
```

```
    match p:
        case ['u', j, t, q]:
            need(type(j) is int and 0 <= j < n)
            syntax(t, n)
            u, B = g[j]
            if u:
                need(prove(q, g) == u)
            A = sub(B, t)
        case ['e', q, r]:
            D, = prove(q, g)
            B = sub(D, b, 1)
            u, E = (sub(B, 0), sub(B, [0, 0, 0]))
            t0 = E[1] if u == [0, 0] and E[0] == b else b
            B = walk(B, lambda i, d: t0 if i == b else i)
            A = prove(r, g + [(u, B)])
        case ['i', B0, t, r]:
            syntax(t, n)
            B = sub(B0, t, 1)
            u = sub(B, b if len(B0) == 2 and (not t) else 0)
            need(prove(r, g + [([], u)]) == sub(B, [0, b, 0]))
            A = [B0]
        case _:
            need(False)
    syntax(A, n, True)
    return A

def check(data):
    need(set(data) == {'goal', 'proof'})
    syntax(data['goal'], 1, True)
    need(prove(data['proof'], [([], [1, 1])]) == data['goal'])
if __name__ == '__main__':
    check(json.load(open(0)))
    print('ACCEPT')
```

B Contents and reproduction

The archive contains this source and its compiled PDF, the complete checker, the unchanged comparison checker, positive and negative certificates, the scalar-to-R proof translator and direct arithmetic macro builders used in testing, the reproduction program, metrics, provenance, a claim ledger, and a checksum manifest. The reference code and excerpts retain the repository's MIT No Attribution license.

From the extracted directory:

```
python3 reproduce.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
python3 code/checker.py < code/examples/reflexivity.json
```

The new report and its conclusions are specific to the pinned repository snapshot. No repository mutation is needed to reproduce the result.

References

[1] V. Reshetnikov and ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, README at commit `9f9545f5ffd9e0392d4a36639503aca12383450a`. Pinned project directory.

[2] ProveIt, *Uniform binding*, `Research/uniform_binding/report.tex` and `code/checker.py`, same commit. Source of the counter recursor, decoder, direct recursion compiler, and short ordinary logical macros. Pinned package.

[3] ProveIt, *Inferred witness*, `Research/inferred_witness/theory/opening_discharge.tex`, `theory/bridge.tex`, and `code/checker.py`, same commit. Source of inferred opening and its substitution/completeness proof. Pinned package.

[4] ProveIt, `Lean/ConciseIPRA/`, especially `NativeOpenClauses`, `NativeBiInterpretation`, and `FullNormalization`, same commit. These formalize inherited scalar results, not the new counter-aware normalization or new checker in this report. Pinned Lean sources.

[5] ProveIt, `Research/certified_zero/reference/`, `inherited_choice_basis.tex` and `equality_reflection.tex`, same commit. Sources of the logical selector construction and the equation-induction completion argument. Pinned reference directory.

[6] ProveIt, `Research/canonical_recursors/README.md`, same commit. The 516-byte compact profile and its separate public syntax validation contract. Pinned README.

[7] ProveIt, `Research/checked_refinement/`, `code/checker_native.py` and `CLAIMS.json`, same commit. The 788-byte native public-goal profile. Pinned package.

[8] ProveIt, `Research/deterministic_specialization/`, `supplement/scope_fusion/checker.py`, same commit. The supplementary 820-byte Eq/Q/D validator profile. Pinned package.